

Universidad Tecnológica Nacional

Facultad Regional Avellaneda

Departamento de Ingeniería Electrónica

Cátedra: Técnicas Digitales III

Título del Trabajo Práctico:	Trabajo Práctico INTEGRADOR: Contenidos analizados: • Cap1: Manejo de Tareas • Cap2: Manejo de Colas • Cap3: Gestión de Interrupciones / Semáforos • Cap4: Administración de Recursos / Exclusión • Cap5: Uso de Memoria (Contenidos Integrados)
Autores:	Mehle Ignacio
	Eirea Alejandro
IMPORTANTE: <u>Condición de Aprobación (Nota: 4 o 5):</u> Sujeta a funcionamiento general del equipo con base en su funcionamiento mínimo y exposición oral. <u>Condición de Promoción (Nota: 6 - 10):</u> Tener el 100% de la consigna desarrollada con informe y presentación oral. Todos los integrantes del grupo DEBEN estar presentes el día de la entrega del Parcial, de no ser así el parcial se debe recuperar en la instancia de recuperación.	

Sistemas Operativos en Tiempo Real: FREERTOS (Trabajo Práctico INT)

Parte Teórica:

Consigna: se deberá adjuntar todas las partes Teóricas; con sus consignas, de cada Trabajo Práctico anterior.

NOMBRE Y APELLIDO	CALIFICACIÓN EXPOSICIÓN ORAL
1.- Mehle Ignacio	
2.- Eirea Alejandro	

TP1 - TAREAS

1. Sistemas Operativos – Generalidades

a. ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?

El Planificador o *Scheduler* es el encargado de administrar el tiempo que la CPU le dedica a una determinada tarea o *Task*.

Una tarea puede estar ejecutándose (Running) o no ejecutándose (Not Running). Cuando una tarea se encuentra ejecutándose, el procesador está ejecutando su código. Cuando una tarea no está ejecutándose, la tarea está inactiva, su estado ha sido salvado y está listo para retomar su ejecución la próxima vez que el planificador (Scheduler) decida que debe entrar en el estado de ejecución. El Scheduler es la única entidad capaz de intercambiar los estados de las tareas (running/not running).

Asimismo, el Scheduler asegurará que siempre la tarea de mayor prioridad que esté disponible para ser ejecutada será la que entre en el estado de ejecución. Cuando más de una tarea con el mismo nivel de prioridad esté disponible para ser ejecutada, el Scheduler alternará cada tarea, ejecutando una a la vez.

b. Explique a que se denomina “Contexto de ejecución”. ¿Cuáles son las variables intervinientes?

A medida que se ejecuta una tarea, utiliza los registros de la CPU, el Stack Pointer, el Instruction Pointer y el contenido del Stack. Estos recursos juntos (los registros del procesador, la pila, etc.) comprenden el contexto de ejecución de la tarea.

Una tarea es una pieza secuencial de código: no sabe cuándo va a ser suspendida o reanudada por el Kernel, y ni siquiera sabe cuándo ha sucedido esto. Tomemos el ejemplo de una tarea que se suspende inmediatamente antes de ejecutar una instrucción que suma los valores contenidos en dos registros del procesador. Mientras la tarea está suspendida, otras tareas se ejecutarán y pueden modificar los valores del registro del procesador. Al reanudarse, la tarea no sabrá que los registros del procesador han sido alterados; si utiliza los valores modificados la suma daría como resultado un valor incorrecto.

Para evitar este tipo de error es esencial que al reanudarse una tarea tenga un contexto idéntico al inmediatamente anterior a la de su suspensión. El Kernel del sistema operativo es responsable de garantizar que este sea el caso, y lo hace guardando el contexto de una tarea cuando está suspendida. Cuando se reanuda la tarea, el Kernel del sistema operativo restaura su contexto guardado antes de su ejecución. El proceso de guardar el contexto de una tarea que se suspende y restaurar el contexto de una tarea que se reanuda es llamado cambio de contexto.

c. ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

Un sistema operativo de tiempo real (RTOS) está diseñado para ejecutar varias tareas de forma simultánea (multitasking) con el agregado de responder de forma rápida a eventos del mundo real, en un plazo establecido de tiempo, por lo que se llaman “determinísticos”. Esta es la principal ventaja de los RTOS, ya que pueden emplearse en sistemas que respondan a eventos en un tiempo crítico, como el airbag de un auto. En comparación a los sistemas *bare metal*, el Scheduler se encarga de la gestión de las tareas de acuerdo a su prioridad, aligerando la carga lógica por parte del

programador. Por otro lado, los RTOS poseen un tamaño reducido, lo que los hace ideales para dispositivos pequeños como los microcontroladores, frente a los sistemas operativos de propósito general (Windows, Linux, etc.).

2. Kernel de FreeRTOS

a. Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

Las tareas (Tasks) son implementadas como funciones en C. Cada tarea es un pequeño programa, tienen un punto de entrada y cada una corre en un bucle infinito del que nunca sale.

Las tareas de FreeRTOS no deben poder retornar de su función de implementación en ningún caso, no tienen una declaración de retorno y no se debe permitir que se ejecute pasado el final de la función. Si una tarea no se requiere más, debe ser explícitamente borrada.

Una sola definición de una función de tarea puede usarse para crear cualquier número de tareas. Cada tarea creada es una instancia separada de ejecución, con su propia pila y su propia copia de variables definida en la tarea misma. Para crear una tarea se utiliza la función `xTaskCreate()`:

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        configSTACK_DEPTH_TYPE usStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask);
```

Cuyos parámetros son:

- `pvTaskCode`: Puntero a la función que implementa la tarea.
- `pcName`: Nombre descriptivo de la tarea (debug).
- `usStackDepth`: Tamaño de la pila que se asigna a esa tarea.
- `pvParameters`: Parámetros que se le pasan a la tarea.
- `uxPriority`: Prioridad de ejecución de la tarea.
- `pxCreatedTask`: “Handle” de la tarea creada (referencia a la tarea).

Ejemplo de una tarea que parpadea un led (“ blinky ”):

```
static void Blinky( void *pvParameters ){

    // Configuraciones, corren una sola vez
    char delay_ms = 500; //Duración del delay

    // Bucle infinito de la tarea
    while(1){
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        vTaskDelay( delay_ms / portTICK_PERIOD_MS);
    }
}
```

b. Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

Si bien el multitasking hace parecer que la CPU ejecuta tareas simultáneamente, el procesador solo está ejecutando una tarea a la vez en cada instante de tiempo. Esta tarea que se está ejecutando en un preciso momento se dice que está en estado RUNNING, lo que significa que el procesador está ejecutando su código, mientras que todas las demás tareas que no se están ejecutando se encuentran en estado NOT RUNNING. El problema es que, si todas las tareas se encuentran disponibles para ser ejecutadas, las tareas de mayor prioridad impiden que las de menor prioridad entren en ejecución.

Para que esto no ocurra, las tareas deben de poder ser manejadas por eventos. Una tarea manejada por un evento solo tiene trabajo (proceso) que realizar luego que haya ocurrido el evento que la dispare, y no puede entrar en el estado de ejecución antes que dicho evento ocurra. El Scheduler siempre selecciona la tarea de mayor prioridad que se encuentre disponible para ser ejecutada. Si la tarea de mayor prioridad no se encuentra disponible para ser ejecutada, el Scheduler seleccionará la siguiente tarea con mayor prioridad que si se encuentre disponible. Usar tareas manejadas por eventos significa entonces que las tareas pueden ser creadas en diferentes prioridades, sin que la tarea de mayor prioridad impida que las tareas de menor prioridad alguna vez se ejecuten. De esta forma, aparecen 3 sub estados del estado de no ejecución: READY, SUSPENDED y BLOCKED.

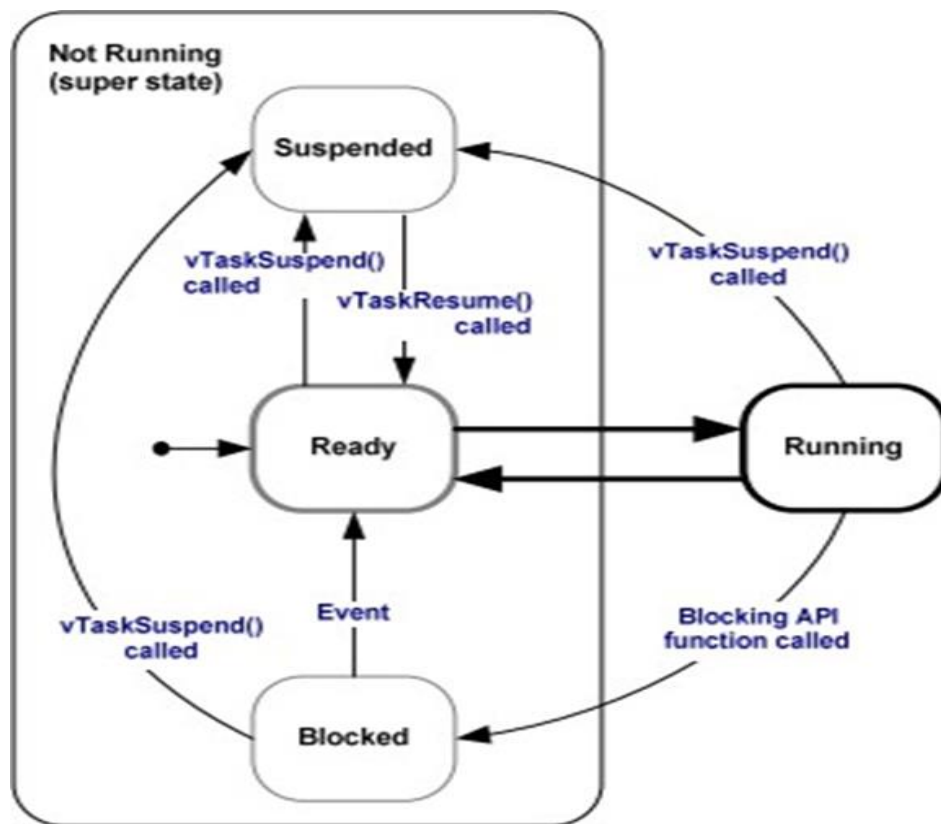
Una tarea que está esperando que ocurra un evento se dice que se encuentra en estado BLOCKED (bloqueada), y no puede ejecutarse hasta salir de ese estado. Las tareas ejecutándose pueden ser bloqueadas por funciones de la API y deben esperar dos tipos de eventos para salir hacia el estado READY:

- Temporales: Debe cumplirse un retardo de tiempo.
- De sincronización: El evento es generado desde otra tarea o interrupción.

Las tareas en el estado SUSPENDED (suspendidas) no están disponibles para que el Scheduler las ejecute. La única manera que una tarea entre en este estado desde cualquiera de los otros 3 es mediante un llamado a la función API `vTaskSuspend()`, y la única manera de salir de este estado es hacia el estado READY mediante un llamado a la función `vTaskResume()` o `vTaskResumeFromISR()`.

Las tareas que no se están ejecutando, pero no están bloqueadas ni suspendidas, están en estado READY (disponible). Pueden ser ejecutadas, están “listas” para hacerlo, pero aún no fueron elegidas por el Scheduler para ser ejecutadas. El Scheduler siempre selecciona la tarea de mayor prioridad que se encuentre disponible para ser ejecutada. Si la tarea de mayor prioridad no se encuentra disponible para ser ejecutada, el Scheduler seleccionará la siguiente tarea con mayor prioridad que si se encuentre disponible.

En el diagrama de transiciones de estados, la doble flecha representa el Scheduler. Como aclaración, todas las tareas arrancan corriendo en READY, pero nunca pueden pasar de READY a BLOCKED.



**c. ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”?
¿Y a qué nos referimos con el concepto de “tareas manejadas por eventos”?**

Las tareas de *procesamiento continuo* son aquellas que no hacen ningún llamado a funciones API que pudieran hacer que las tareas se bloqueen, por lo que se encontrarán siempre ejecutándose o disponibles para ejecutarse. Estas tareas siempre tienen trabajo que realizar, aunque sea un trabajo trivial en este caso.

Por otro lado, como se mencionó anteriormente, una tarea *manejada por un evento* solo tiene trabajo (proceso) que realizar luego que haya ocurrido el evento que la dispara, y no puede entrar en el estado de ejecución antes que dicho evento ocurra.

d. ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Las tareas pueden entrar en el estado bloqueado para esperar por dos tipos distintos de eventos:

- **Eventos Temporales:** el evento consta de un cierto retardo de tiempo que debe cumplirse o un tiempo absoluto que debe alcanzarse. Por ejemplo, una tarea puede bloquearse llamando a la función `vTaskDelay (500/ portTICK_PERIOD_MS)` para esperar que pasen 500 milisegundos hasta que vuelva a estar disponible.
- **Eventos de Sincronización:** donde el evento es originado desde otra tarea o interrupción. Por ejemplo, una tarea puede bloquearse para esperar que ingresen datos en una cola. Los eventos de sincronización cubren un amplio rango de tipos de eventos.

e. ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

El procesador siempre necesita algo para ejecutar, es decir, debe haber siempre al menos una tarea que puede entrar en el estado de ejecución. Para asegurarse de esto, una tarea ociosa (IDLE TASK) se crea automáticamente por el Scheduler cuando se llama a la función `vTaskStartScheduler ()`. La tarea ociosa no hace más que entrar en un bucle, y siempre es capaz de ejecutarse, además, tiene la prioridad más baja posible (prioridad 0) para asegurarse que no impida que una tarea de mayor prioridad entre en el estado de ejecución.

.

TP2 – COLAS

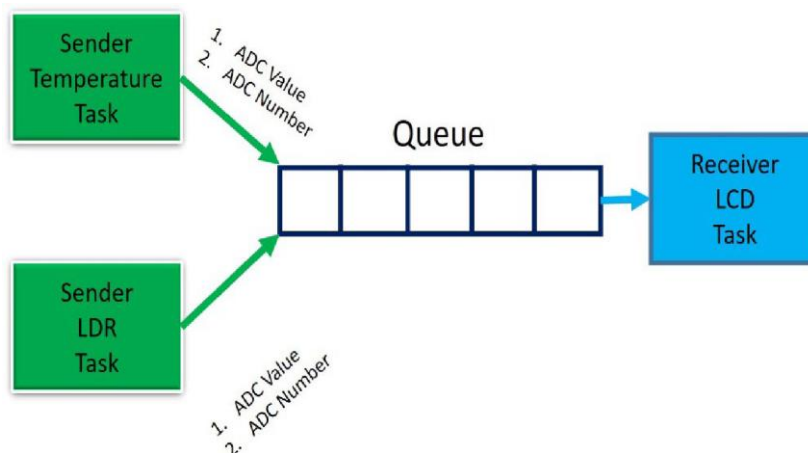
3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

Las colas son la forma más básica de comunicación entre tareas de FreeRTOS. Pueden ser usadas para enviar datos entre tareas o entre interrupciones y tareas. Elementalmente toman la forma de buffers FIFO (**First In First Out**), donde los datos se escriben al final de la cola, para ser leídos al frente de la misma, aunque con la flexibilidad de que los datos pueden sobre escribirse al frente.

Las colas pueden ser leídas y escritas por múltiples tareas, pero solo puede hacerlo una tarea a la vez. Si varias tareas quieren leer/escribir una cola, solo puede hacerlo la tarea de mayor prioridad, el resto de tareas se bloquearán.

4) Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora.

En un panorama donde una tarea receptora requiera recibir información de varias tareas, como las colas no están asignadas a una tarea en particular, es mucho más sencillo utilizar una sola cola con múltiples tareas que escriban en ella, que una cola individual para cada una de las tareas. Para que la tarea receptora sepa de donde proviene cada dato de la cola, se definen los ítems de la cola como estructuras, con un campo que contenga el dato, y otro que contenga el contenido/significado del dato, o de que tarea proviene. Por ejemplo, la tarea que maneja el stack FreeRTOS-Plus-UDP IP emplea una sola cola para recibir notificaciones de eventos del protocolo ARP, paquetes recibidos del hardware Ethernet, paquetes recibidos desde la aplicación, eventos de caída de la red, etc.



Se presenta el ejemplo de la imagen, donde dos tareas envían datos a la misma cola, para ser leída por una sola tarea receptora:

```
#include "main.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

// include libreria LCD
#include<LiquidCrystal.h>

// DEFINICIONES
typedef struct
{
    uint32_t pin; // Pin leído
    uint32_t value; // Valor leído del adc
```



```

} pinRead_t;

QueueHandle_t structQueue; // Handle de la cola

ADC_HandleTypeDef hadc0; // Handle del ADC0
ADC_HandleTypeDef hadc1; // Handle del ADC1

/* Private function prototypes
   */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC0_Init(void);
static void MX_ADC1_Init(void);

// FUNCIONES

// Lectura del sensor de temperatura
static void TaskTempReadPin0(void *pvParameters){
    while(1)
    {
        pinRead_t currentPinRead; // Defino un buffer pinRead_t
        currentPinRead.pin = 0; // Pin leido = 0

        // Leo la entrada 0 del ADC y lo guardo en el campo value
        HAL_ADC_Start(&hadc0);
        while(HAL_ADC_PollForConversion(&hadc0, 1000) != HAL_OK);
        currentPinRead.value = HAL_ADC_GetValue(&hadc0);
        // Escribo el buffer en la cola
        xQueueSend(structQueue, &currentPinRead, portMAX_DELAY);
        taskYIELD(); // Termino la tarea e informo al scheduler que switchee
    }
}

// Lectura del sensor de lux
static void TaskLightReadPin1(void *pvParameters){
    while(1)
    {
        pinRead_t currentPinRead; // Defino un buffer pinRead_t
        currentPinRead.pin = 1; // Pin leido = 0
        // Leo la entrada 1 del ADC y lo guardo en el campo value
        HAL_ADC_Start(&hadc1);
        while(HAL_ADC_PollForConversion(&hadc1, 1000) != HAL_OK);
        currentPinRead.value = HAL_ADC_GetValue(&hadc1);
        // Escribo el buffer en la cola
        xQueueSend(structQueue, &currentPinRead, portMAX_DELAY);
        taskYIELD(); // Termino la tarea e informo al scheduler que switchee
    }
}

// Lectura de la cola y muestro temperatura y lux en filas del LCD
static void TaskLcd(void * pvParameters){
    while(1)
    {
        uint32_t temperatura = 0; // Variable aux que guarde la temperatura
        uint32_t lux = 0; // Variable aux que guarde los lux
        pinRead_t currentPinRead; // Buffer de lectura de la cola
    }
}

```

```

// Leo un elemento de la cola y verifico lectura correcta
if (xQueueReceive(structQueue, &currentPinRead, portMAX_DELAY) ==
pdPASS)
{
    // Si el dato es del ADC0, es un valor de temperatura
    if(currentPinRead.pin == 0)
    {
        // Convierto la muestra del ADC0 en un valor de temperatura
        temperatura = (uint32_t)(currentPinRead.value * 500/4096);

        // Muestro en la primera linea del LCD
        lcd.setCursor(0, 0);
        lcd.print("Temp = ");
        lcd.setCursor(7, 0);
        lcd.print(temperatura);
        lcd.print("'C");
    }

    // Si el dato es del ADC1, es un valor de lux
    if(currentPinRead.pin == 1)
    {
        // Convierto la muestra del ADC1 en un valor de lux
        lux = map(currentPinRead.value, 0, 4095, 0, 255);
        // Muestro en la segunda linea del LCD
        lcd.setCursor(0, 1);
        lcd.print("LIGHT = ");
        lcd.setCursor(7, 1);
        lcd.print(lux);
        lcd.print("LUX");
    }
}
taskYIELD(); // Termino la tarea e informo al scheduler que switchee
}

void main()
{
    /* Reset of all peripherals, Initializes the Flash interface and the
    SysTick. */
    HAL_Init();
    /* Configure the system clock */
    SystemClock_Config();
    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_ADC0_Init();
    MX_ADC1_Init();

    // Defino pines del LCD
    LiquidCrystal lcd(7,6,5,4,3,2); // RS, E, D4, D5, D6, D7
    lcd.begin(16, 2); // Enable LCD library

    // Creo una cola de elementos pinRead_t
    structQueue = xQueueCreate(10, sizeof(pinRead_t));
    if(structQueue != NULL){
        // Creo una tarea que muestre los datos en el LCD

```

```

xTaskCreate(TaskLcd, "Displaydata", 128, NULL, 2, NULL);

// Creo una tarea que lea el sensor de temperatura en el pin ADC0
xTaskCreate(TaskTempReadPin0, "AnalogReadPin0", 128, NULL, 1, NULL);

// Creo una tarea que lea el sensor de lux en el pin ADC1
xTaskCreate(TaskLightReadPin1, "AnalogReadPin1", 128, NULL, 1, NULL);
}
vTaskStartScheduler();
}

```

Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

Cuando una tarea intenta **leer** de una cola vacía, la misma se *bloquea*, y en el caso de que ocurra esto es donde se puede especificar un “tiempo de bloqueo”. Este es el tiempo en que la tarea debe mantenerse en el estado *bloqueado* esperando a que los datos estén disponibles en la cola. Cuando otra tarea o interrupción coloca datos en la cola, la tarea que intentó leer vuelve automáticamente a estado *ready*. También pasa a estado *ready* si el tiempo de bloqueo expira antes de que los datos sean escritos en la cola.

Si una cola tiene más de una tarea bloqueada esperando por los datos, la tarea que se desbloquea será siempre la de mayor prioridad que estaba esperando por los datos. Si las tareas bloqueadas tienen igual prioridad, entonces la tarea que ha estado esperando más tiempo por el dato será la que se desbloquea.

Cuando una tarea intenta **escribir** en una cola llena, la misma se *bloquea*, y en el caso de que ocurra esto es donde se puede especificar un “tiempo de bloqueo”. Este es el tiempo en que la tarea debe mantenerse en el estado *bloqueado* esperando a que vuelva a haber espacio disponible en la cola para escribir. Cuando otra tarea o interrupción lee datos de la cola, liberando espacio en ella, la tarea que intentó escribir vuelve automáticamente a estado *ready*. También pasa a estado *ready* si el tiempo de bloqueo expira antes de que la cola vuelva a estar disponible para escritura.

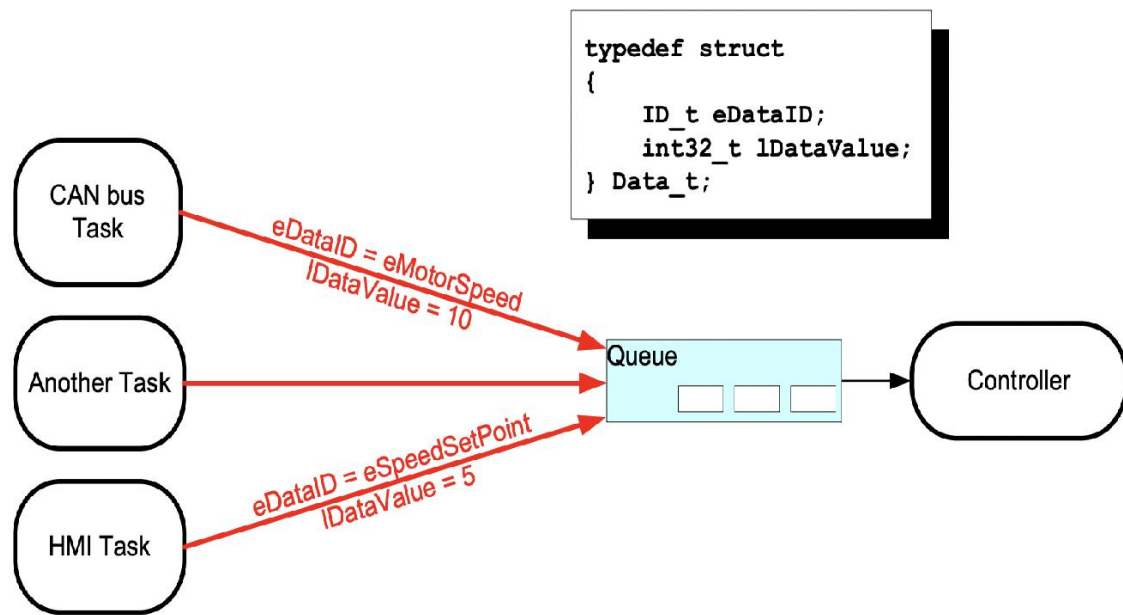
Si una cola tiene más de una tarea bloqueada esperando para escribir en ella, la tarea que se desbloquea será siempre la de mayor prioridad que estaba esperando para escribir datos. Si las tareas bloqueadas tienen igual prioridad, entonces la tarea que ha estado esperando más tiempo por espacio disponible será la que se desbloquea.

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM?

Para enviar datos compuestos, como en el caso de múltiples tareas enviando datos a una única tarea receptora del punto (2), se definen los ítems de la cola como estructuras conteniendo un campo con el dato en cuestión, y otro campo que especifique a que tarea corresponde ese dato enviado. En este caso, para definir el tamaño de cada ítem en el parámetro *uxItemSize* se puede emplear la función nativa de C *sizeof()* para obtener el tamaño de la estructura.

Un caso interesante se da cuando los datos que debe contener la cola son grandes (arrays, strings, etc.), una forma de ahorrar memoria RAM es pasar un puntero a estos datos a la cola, pero para emplear esta metodología deben tomarse recaudos. Por un lado, es deseable que solamente la tarea de enviar tenga permitido acceder a la memoria hasta que un puntero a la memoria haya sido cargado en cola, y sólo la tarea de recepción debería permitir acceder a la memoria después de que el puntero se ha recibido de la cola. Si la memoria siendo apuntada fue asignada dinámicamente entonces sólo una tarea debe ser responsable de liberar la memoria. Ninguna tarea debería tratar de acceder a la memoria después de que haya sido liberada. Por último, un puntero nunca debe ser usado para acceder a datos que han sido asignados en una pila de tareas, ya que los datos no serán válidos después de que la pila haya cambiado.

6) Explicar el funcionamiento del siguiente diagrama:



En el diagrama se muestra el uso de una estructura de datos `Data_t` definida para agrupar 2 campos necesarios para el controlador. Estos son `eDataID` y `lDataValue`. Esto se hace para tener un tipo de variable nueva y definida por el usuario que es común a las tareas utilizadas. Esto es: la tarea CAN bus Task usa el tipo de dato `Data_t` para informar respectivamente su identificación de dato (velocidad de motor: `eMotorSpeed`) y la cantidad informada en el otro campo: `lDataValue`: 10. Esto es escrito en una cola mostrada genéricamente como Queue. Luego se muestra una tarea genérica: Another Task escribiendo en la misma tarea y finalmente se muestra otra tarea específica: HMI Task en donde se usa el mismo tipo de dato pero para informar en este caso velocidad requerida y valor (`eDataID`: `eSpeedSetPoint` y `lDataValue` = 5) respectivamente. De esta manera se muestra que cuando múltiples tareas deben informar el mismo tipo de dato pero de diferente característica, conviene hacer una estructura que englobe estos datos, y se muestra como múltiples tareas escriben en una cola y luego ésta es leída por otra (tarea del controlador).

TP3 – SEMÁFOROS Y ADMINISTRACION DE RECURSOS

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

Un Mutex (Mutual Exclusion Semaphore) es un tipo especial de semáforo que sirve para garantizar acceso exclusivo a un recurso compartido (por ejemplo: un periférico, una variable global, un archivo en memoria, etc.) Aunque se parece a un semáforo binario, tiene dos diferencias clave:

- Herencia de prioridad: si una tarea de baja prioridad tiene el mutex y una tarea de alta prioridad lo quiere usar, FreeRTOS eleva temporalmente la prioridad de la tarea de baja para evitar inversión de prioridades.
- Dueño del recurso: el mutex debe ser tomado (taken) y liberado (given) por la misma tarea.

Se muestra un código ejemplo en donde se comparte el bus de I²C por dos tareas por medio de un mutex.

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "queue.h"
```

```

#include "helper.h" //libreria para PWM
#include "lcd.h" //libreria de LCD
#include "bmp280.h" //libreria del sensor de presion y temperatura BMP280

#define I2C_PORT i2c0
#define I2C_SDA 8 //PIN 11-GPIO8
#define I2C_SCL 9 //PIN 12-GPIO9
#define ADDR_LCD 0x27 // Direccion de 7 bits del adaptador del LCD

SemaphoreHandle_t Mutex_I2C; //Handler del semaforo mutex para el I2C
QueueHandle_t Queue_data; // Handler de la cola

//Estructura para manejo de los datos del sensor
typedef struct {
    float temp;
    float presion;
} datasensor_t;

datasensor_t datos_bmp; //Estructura de datos para la calibracion del sensor y calculos

//TAREA LECTURA SENSOR TEMPERATURA Y PRESION
void task_BMP280(void *params)
{
    // Obtiene los parámetros de calibración
    struct bmp280_calib_param struct_calib_params;
    xSemaphoreTake(Mutex_I2C ,portMAX_DELAY); //Toma el semaforo para que ningun otra tarea
pueda utilizar el bus I2C
    bmp280_get_calib_params(&struct_calib_params); //obtiene los parámetros de calibracion del sensor
    xSemaphoreGive(Mutex_I2C); //Devuelve el semaforo I2C

    while (1){
        int32_t raw_temp, raw_presion;
        xSemaphoreTake(Mutex_I2C ,portMAX_DELAY); //vuelve a tomar el semaforo ahora para obtener
los datos RAW del sensor
        bmp280_read_raw(&raw_temp, &raw_presion); //Obtiene ambos valores sin compensar
        xSemaphoreGive(Mutex_I2C); //Devuelve semaforo

        datos_bmp.temp = bmp280_convert_temp(raw_temp, &struct_calib_params); //
Convierte temperatura
        datos_bmp.presion = bmp280_convert_pressure(raw_presion, raw_temp, &struct_calib_params) /
100.00; // Convierte presion

        printf("Temperatura: %.2f °C \nPresion: %.2f hPa\n", datos_bmp.temp, datos_bmp.presion);
//Imprime datos por consola serial para debuggear

        xQueueOverwrite(Queue_data, &datos_bmp); //Sobreescribe datos del sensor en cola para escribir en
el LCD.

        vTaskDelay(pdMS_TO_TICKS(500)); //Delay de 100ms hasta proxima lectura
    }
}

//RECIBO DE DATOS DE LA COLA E IMPRESIÓN EN EL LCD
void task_LCD(void *params)
{
    datasensor_t datos_LCD;

    // Variable para imprimir el mensaje

```

```

char str[16];
float error_prueba;

while (1){
    xQueuePeek(Queue_data, &datos_LCD, portMAX_DELAY);    //Copia Queue_data en datos_LCD
    xSemaphoreTake(Mutex_I2C ,portMAX_DELAY);              //Toma el semaforo mutex para poder
imprimir en LCD y no ser interrumpido por lectura de datos del sensor

    lcd_set_cursor(0, 0);
    sprintf(str, "Temp: %.2f C", datos_LCD.temp);
    lcd_string(str);
    lcd_set_cursor(1, 0);
    sprintf(str,"Presion: %.2f hPa", datos_LCD.presion);
    lcd_string(str);
    printf("ESTOY ACA\n");
    xSemaphoreGive(Mutex_I2C);          //Devuelve semaforo mutex I2C para libre utilizacion del
sensor, si fuese necesario
    vTaskDelay(pdMS_TO_TICKS(500));    //demora de 100ms hasta proxima secuencia de
impresion
}
}

int main(){

    stdio_init_all();

    //creacion del semaforo mutex
    Mutex_I2C = xSemaphoreCreateMutex();

    //Creacion de la cola que enviara datos de Task_BMP280 a Task_LCD
    Queue_data = xQueueCreate(1, sizeof(datasensor_t));

    i2c_init(I2C_PORT, 400*1000);        // Inicializo el I2C con un clock de 400 KHz
    gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);    // Habilito la funcion de I2C en los GPIOs
    gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
    gpio_pull_up(I2C_SDA);                // Habilito pull-ups
    gpio_pull_up(I2C_SCL);
    lcd_init(I2C_PORT, ADDR_LCD);          // Inicializo LCD
    lcd_clear();

    bmp280_init(i2c0);                    // Inicializa el BMP280 usando el I2C0

    xTaskCreate(task_BMP280, "Task_BMP280", 4*configMINIMAL_STACK_SIZE, NULL, 1, NULL);
    xTaskCreate(task_LCD, "Task_LCD", 4*configMINIMAL_STACK_SIZE, NULL, 1, NULL);

    // Arranca el scheduler
    vTaskStartScheduler();

    while (true);
}

```

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

Concepto de Inversión de Prioridad

La inversión de prioridad ocurre cuando una tarea de alta prioridad necesita un recurso (ej. UART, variable compartida), ese recurso está siendo usado por una tarea de baja prioridad. La tarea de alta prioridad queda bloqueada, esperando a que la de baja termine. Pero si en el medio aparece una tarea de prioridad media, esta sigue ejecutándose porque no está bloqueada, impidiendo que la de baja prioridad libere el recurso. El resultado es que la tarea de alta prioridad queda atascada, esperando indirectamente a una de menor prioridad. Eso rompe la idea de que "la tarea más importante siempre corre primero".

Ejemplo práctico:

- Tarea A → prioridad alta (10). Necesita I2C.
- Tarea B → prioridad baja (1). Tiene el I2C en uso.
- Tarea C → prioridad media (5). No usa I2C.

Explicación del comportamiento:

- B comienza a usar UART.
- A intenta usarlo → queda bloqueada porque el recurso lo tiene B.
- C comienza a ejecutar porque tiene más prioridad que B.
- B no puede liberar el recurso porque nunca se ejecuta.
- A sigue bloqueada, aunque es la más importante. Aquí se invierte la prioridad

Código de Ejemplo:

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"
#include <stdio.h>

QueueHandle_t xQueue;
SemaphoreHandle_t xMutex;

/* -----
Tarea Baja (Productor)
----- */
void TareaBaja(void *pvParameters) {
    int dato = 0;
    while (true) {
        // Toma el mutex
        if (xSemaphoreTake(xMutex, portMAX_DELAY)) {
            dato++;
            printf("Tarea Baja: tengo mutex, escribiendo en cola...\n");

            // Simula trabajo lento
            vTaskDelay(pdMS_TO_TICKS(200));

            // Escribe en la cola
            if (xQueueSend(xQueue, &dato, 0) == pdPASS) {
                printf("Tarea Baja: dato %d enviado a la cola\n", dato);
            }

            // Libera el mutex
            xSemaphoreGive(xMutex);
        }
    }
}
```



```

        printf("Tarea Baja: liberé mutex\n");
    }
    vTaskDelay(pdMS_TO_TICKS(1000));
}

/* -----
Tarea Alta (Consumidor)
----- */
void TareaAlta(void *pvParameters) {
    int recibido;
    while(true) {
        // Quiere el mutex para leer la cola
        if (xSemaphoreTake(xMutex, portMAX_DELAY)) {
            printf("Tarea Alta: tengo mutex, leyendo cola...\n");

            if (xQueueReceive(xQueue, &recibido, 0) == pdPASS) {
                printf("Tarea Alta: recibido %d\n", recibido);
            }

            // Libera el mutex
            xSemaphoreGive(xMutex);
            printf("Tarea Alta: liberé mutex\n");
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

/* -----
main
----- */
int main(void) {
    // Crear cola
    xQueue = xQueueCreate(5, sizeof(int));
    // Crear mutex
    xMutex = xSemaphoreCreateMutex();

    if (xQueue != NULL && xMutex != NULL) {
        // Crear tareas con prioridades distintas
        xTaskCreate(TareaBaja, "Baja", 1000, NULL, 1, NULL); // prioridad baja
        xTaskCreate(TareaAlta, "Alta", 1000, NULL, 3, NULL); // prioridad alta

        // Iniciar scheduler
        vTaskStartScheduler();
    }

    while (true);
}

```

La lógica del ejemplo es que la tarea de baja prioridad toma el mutex y comienza a trabajar (simula escribir en la cola). Justo después, la tarea de alta prioridad intenta tomar el mutex → queda bloqueada esperando. Sin herencia de prioridad: si hubiera otra tarea de prioridad media, esta podría ejecutarse primero y retrasar a la de alta.

Con herencia de prioridad: el scheduler sube la prioridad de la tarea baja para que libere rápido el mutex y la tarea alta corra enseguida.

9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.

Para hablar de Herencia de Prioridad primero debemos explicar el problema que lo trae a cuestión, y este es la Inversión de prioridad.

En un sistema de planificación por prioridades fijas (como FreeRTOS), lo lógico es que la tarea de mayor prioridad se ejecuta primero siempre que esté lista, pero si una tarea de baja prioridad o de prioridad menor tiene un recurso, la de alta se bloquea y una de media puede “colarse”, creando la inversión de prioridad.

Cuando se usa un mutex en FreeRTOS, automáticamente se activa la herencia de prioridad. Una tarea de baja prioridad (P1) toma el mutex, una tarea de alta prioridad (P10) intenta tomarlo pero queda bloqueada. FreeRTOS detecta que la de baja está retrasando a la de alta, entonces sube temporalmente la prioridad de la baja hasta la prioridad de la alta (P10). Ahora la de baja se ejecuta sin ser interrumpida por tareas medias y cuando libera el mutex, FreeRTOS le devuelve su prioridad original (P1).

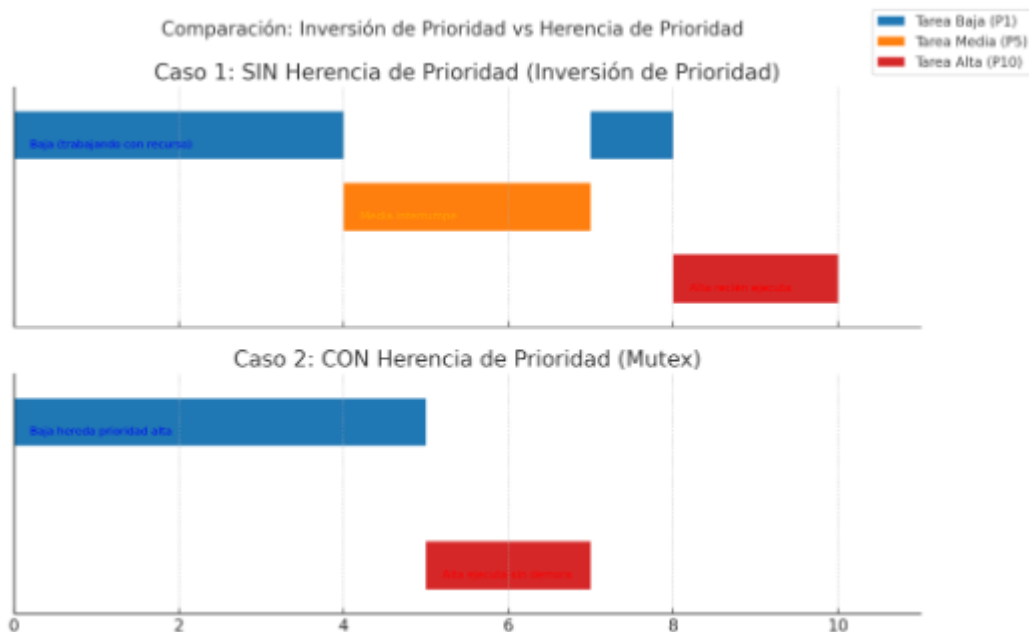
Con esto se consigue que una tarea de media prioridad (P5) cause un retraso, se garantiza que el recurso se libere lo antes posible y se conserva el determinismo en sistemas de tiempo real

.Ejemplo práctico con herencia de prioridad:

- Tarea Baja (P1) tiene el bus de I2C.
- Tarea Alta (P10) lo pide → se bloquea.
- FreeRTOS sube la prioridad de la Tarea Baja a P10.
- Ahora la Tarea Baja ejecuta rápido, libera el I2C.
- Tarea Alta toma el recurso.
- Tarea Baja vuelve a P1.

En este caso, aunque llegue la tarea Media (P5), no interrumpe porque la Tarea Baja ahora **“heredó”** P10.

Gráficamente:



10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

Concepto de Punto Muerto (Deadlock)

Un punto muerto ocurre cuando dos o más tareas quedan bloqueadas indefinidamente, porque cada una está esperando un recurso que la otra posee.

Para que exista, se deben cumplir las siguientes condiciones:

- Exclusión mutua: los recursos no pueden compartirse, solo una tarea los usa a la vez (ej: un mutex).
- Retención y espera: una tarea tiene un recurso y al mismo tiempo espera por otro.
- No expropiación: los recursos no se pueden quitar a la fuerza; solo el dueño puede liberarlos.
- Espera circular: existe un ciclo de tareas donde cada una espera un recurso que otra tiene.

Ejemplo práctico:

La Tarea A toma el mutex de la UART y luego quiere el mutex del I2C.

La Tarea B toma primero el mutex del I2C y luego quiere el mutex de la UART.

y ocurre que: A espera a que B libere I2C y B espera a que A libere UART. Si ninguna tarea libera los recursos se produce un deadlock total.

Para prevenir puntos muertos en FreeRTOS se debe cumplir un orden consistente de adquisición de recursos. Todas las tareas deben tomar los mutex en el mismo orden (ej: primero UART, luego I2C) así se evita la espera circular. No tener tiempos de espera infinitos, esto es usar `xSemaphoreTake(mutex, timeout)` en vez de esperar infinito (`portMAX_DELAY`). Evitar múltiples recursos al mismo tiempo: Siempre que se pueda, que las tareas solo usen un mutex a la vez, y tener un diseño cuidadoso de prioridades: Reducir la cantidad de tareas compitiendo por varios recursos críticos.

Código de ejemplo:

```
#include "FreeRTOS.h"
```

```
#include "task.h"
```

```
#include "queue.h"
```

```
#include "semphr.h"
```

```
#include <stdio.h>
```

```
QueueHandle_t xQueue;
```

```
SemaphoreHandle_t xMutex;
```

```
/* -----
```

```
Tarea Productor
```

```
----- */
```

```
void TareaProductor(void *pvParameters) {
```

```
    int dato = 0;
```

```
    while(true) {
```

```
        // Primero toma el mutex
```

```
        if (xSemaphoreTake(xMutex, portMAX_DELAY)) {
```

```
            printf("Productor: Tengo el mutex, preparando dato...\n");
```

```

    dato++;

    // Intenta enviar a la cola
    printf("Productor: esperando espacio en cola...\n");
    if (xQueueSend(xQueue, &dato, portMAX_DELAY) == pdPASS) {
        printf("Productor: Enviado dato %d a la cola\n", dato);
    }

    // No libera el mutex hasta que termine
    printf("Productor: Libero el mutex\n");
    xSemaphoreGive(xMutex);
}
vTaskDelay(pdMS_TO_TICKS(500));
}
}

/* -----
Tarea Consumidor
----- */
void TareaConsumidor(void *pvParameters) {
    int recibido;
    while (true) {
        // Primero recibe de la cola
        printf("Consumidor: esperando dato en cola...\n");
        if (xQueueReceive(xQueue, &recibido, portMAX_DELAY) == pdPASS) {
            printf("Consumidor: recibió dato %d\n", recibido);

            // Ahora intenta tomar el mutex
            if (xSemaphoreTake(xMutex, portMAX_DELAY)) {
                printf("Consumidor: tengo mutex, procesando dato %d\n", recibido);
                vTaskDelay(pdMS_TO_TICKS(200));
                xSemaphoreGive(xMutex);
            }
        }
    }
}

/* -----
main
----- */
int main(void) {

```

```
// Crear cola con espacio para 1 elemento
xQueue = xQueueCreate(1, sizeof(int));
// Crear mutex
xMutex = xSemaphoreCreateMutex();

if (xQueue != NULL && xMutex != NULL) {
    // Crear tareas
    xTaskCreate(TareaProductor, "Productor", 1000, NULL, 2, NULL);
    xTaskCreate(TareaConsumidor, "Consumidor", 1000, NULL, 2, NULL);

    // Iniciar scheduler
    vTaskStartScheduler();
}

while (true);
}
```

Ocurre lo siguiente: La tarea Productor toma el mutex y quiere mandar a la cola. La cola está llena o bloqueada → Productor espera que el Consumidor saque un dato. Consumidor recibe de la cola, pero antes de procesar necesita el mutex → queda bloqueado esperando al Productor. Ambos esperan indefinidamente → Deadlock.
